



UNIVERSITY OF PISA

Master's Degree Computer Engineering

Foundations of Cybersecurity Course

BullettinBoardSystem

Work Group:
Giovanni Enrico Loni
Lorenzo Mancinelli

ACADEMIC YEAR 2023/2024

CONTENTS

CONTENTS	i
1 APPLICATION OVERVIEW AND REQUIREMENTS	1
1.1 OVERVIEW	1
1.2 MAIN FUNCTIONS AND OPERATIONS	1
1.2.1 Functions available	1
1.2.2 Operations	2
1.3 REQUIREMENTS	2
2 APPLICATION DESIGN	3
2.1 SERVER	3
2.1.1 Main functions	4
2.2 WORKER	7
2.2.1 Main functions	8
2.3 CLIENT	11
3 SECURITY PROTOCOLS	12
3.0.1 Assumptions and Hypothesis	12
3.1 SESSION ESTABLISHMENT PROTOCOL	12
3.1.1 Protocol description	12
3.1.2 Protocol implementation	13
3.2 SESSION MESSAGE PROTOCOL	22
3.2.1 HMAC generation protocol	22
3.2.2 Message structure	23
3.2.3 Buffer length management	24
3.3 LOGIN PHASE PROTOCOL	24
3.3.1 Password management	25
3.4 REGISTRATION PHASE PROTOCOL	27
3.4.1 Challenge generator	28

4	APPLICATION PROTOCOLS.	31
4.1	LIST() OPERATION.	31
4.2	ADD() OPERATION.	32
4.3	GET() OPERATION.	32

1 APPLICATION OVERVIEW AND REQUIREMENTS

1.1 OVERVIEW

A **BullettinBoardSystem** (BBS) is a distributed service where users can read messages, and add their own.

In BBS, every user is identified by means of a *nickname*, that is established at registration time, together with a password.

A **message** is a tuple composed by the following fields: identifier (*id*), title, author and body. The *id* field uniquely identifies the message within the BBS, and the *author* specifies the nickname of the user which added the message in the system.

1.2 MAIN FUNCTIONS AND OPERATIONS

1.2.1 Functions available

The BBS provides users with the followings functions:

- `List(int n)` which lists the latest *n* available messages in the system;
- `Get(int mId)` which downloads from the BBS the message specified by the input *mId*;
- `Add(string title, string author, string body)` which adds a message to the BBS.

Registered users may issue operations after successful login, and all the operations are executed over a secure channel.

A user who logs out cannot perform operations until they successfully logs in again.

BBS is implemented in a centralized way by a BBS server, which is attested at a well-know (ip, port) couple. Furthermore, the BBS server is equipped by a *private-public* key pair, which public component is know to users.

1.2.2 Operations

Registration phase

- A user securely connects to the BBS server, and specifies an email address, a nickname and a password;
- The server sends a challenge to the email address specified by the user, and waits to receive the challenge back;
- If the user correctly returns the challenge to the server, the registration phase concludes successfully, otherwise is aborted.

Login phase

- A user securely connects to the BBS server, and logs in by means of their nickname and password;
- The server lets the user logs in if the submitted nickname and password are correctly verified;
- Upon successful login, a secure connection is established until the user logs out.

1.3 REQUIREMENTS

- Never store or transmit passwords in the clear;
- Fulfill confidentiality, integrity, no-replay and no-malleability in communications;
- Guarantee Perfect Forward Secrecy (PFS);
- Reduce code vulnerabilities as much as possible;
- User C or C++ programming language and OpenSSL library, but OpenSSL API TLS cannot be used.

2 APPLICATION DESIGN

2.1 SERVER

A class `Server` has been created to handle the server logic, and an approach based on a thread-pool has been implemented.

```
class Server {
public:
    Server(int port, int workerCount, volatile sig_atomic_t* signal_caught);
    ~Server();
    //logic functions
    void acceptClient();
    void startListening();
    //pointers to other main structures
    FileRWLock* fileLock;
    BulletinBoardSystem* bbs;
    FileRWLock* messageLock;
private:
    int socket;
    int port;
    int workerCount;
    //variables to implement threads
    job_t* jobs;
    std::vector<Worker*> workers;
    std::vector<std::thread> workerThreads;

    volatile sig_atomic_t* signal_caught;
    struct sockaddr_in serverAddress;
    struct sockaddr_in userAddress;
    int userAddressLength;
    int userSocket;
};
```

A struct for jobs has been implemented: this struct is responsible for holding the client requests, having each job responsible for holding a single client request; once the client request is done, the job is free to hold another client request.

```
struct job{
    std::vector<int> queue;
    std::atomic_bool isDone;
    std::mutex mutex;
    std::condition_variable cv;
};
typedef struct job job_t;
```

2.1.1 Main functions

Constructor

In the class constructor, three instances of objects have been created:

- FileRWLock* fileLock, a class used to implement a lock mechanism for both files and variables. In this context, it was used to guarantee atomic operations in the users.txt file;
- BulletinBoardSystem* bbs, which is the class that handle the actual operations that users can perform on messages.
- FileRWLock* messageLock, which implements a lock mechanism on the vector of messages.

```
Server::Server(int port, int workerCount, volatile sig_atomic_t*
    signal_caught) {
    this->port = port;
    this->workerCount = workerCount;
    this->signal_caught = signal_caught;
    jobs = new job_t();
    jobs->isDone = false;

    FileRWLock* fileLock = new FileRWLock("res/users/users.txt");
    this->fileLock = fileLock;

    BulletinBoardSystem* bbs = new BulletinBoardSystem();
    this->bbs = bbs;

    FileRWLock* messageLock = new FileRWLock();
    this->messageLock = messageLock;
    return;}
```

```
void acceptClient()
```

In this function, the thread pool is created, and the `select()` is called to receive clients requests. These requests are then assigned and handled by free threads.

```
void Server::acceptClient() {
    jobs->isDone = false;

    workers.reserve(workerCount);
    workerThreads.reserve(workerCount);

    for (int i = 0; i < workerCount; i++) {
        //new worker
        Worker* worker = new Worker(jobs, fileLock, bbs, messageLock);
        workers.push_back(worker);
        workerThreads.emplace_back([&worker]() { worker->workerMain(); });
    }

    struct sockaddr_in userAddress;
    socklen_t userAddressLength = sizeof(struct sockaddr_in);
    int userSocket = -1;

    while(true){
        //set up the select call
        fd_set readfds;
        FD_ZERO(&readfds);
        FD_SET(socket, &readfds);

        //timer for timeout
        struct timeval tv;
        tv.tv_sec = 1;
        tv.tv_usec = 0;

        //select call
        int retval = select(socket + 1, &readfds, nullptr, nullptr, &tv);

        if(retval == -1){
            perror("Error on select");
            std::cin >> retval;
            exit(1);
        }
        if(retval == 0){
            //timeout
            continue;
        }
        userSocket = accept(socket, (struct sockaddr *) &userAddress,
            &userAddressLength);
    }
}
```



```

    if (userSocket < 0) {
        perror("Error on accept");
        exit(1);
    }

    jobs->queue.push_back(userSocket);

    jobs->cv.notify_one();
}
}

```

More in details:

1. Worker threads are created and associated with the shared `jobs` structure, used to receive and process clients connections. Each worker is then added to the `threads` vector, which will be used to join each running threads when the server is stopped;
2. Inside the main loop, server uses the `select()` function to wait for incoming connections, allowing the server to monitor the listener socket. A **timeout** has been added to introduce a mechanism to catch terminating signals: given the fact that the `accept()` function is *blocking*, timeout ensures that the loop will periodically checks for signals, and prevents the server from being unresponsive. If the `select()` function returns `-1`, it checks if the error is due to a signal interruption and, in that cases, continues with the next iteration. If the same function returns `0`, it means that a timeout occurred, meaning that no incoming connections have been received in that interval of time;
3. The server accepts the incoming connection using the `accept()` function, which returns the socket client for the new connection;
4. That socket is then added to the `queue` in the `job` structure, ensuring thread-safe access by using a `std::mutex`;
5. One waiting worker thread is notified that a new connection is available to be handled, using the `std::condition_variable cv`;
6. The loop continues until the `signal_caught` indicates that server should terminate its operation.

2.2 WORKER

To handle incoming requests and operations from clients, a class `Worker` has been defined.

```
class Worker {
public:
    Worker(job_t* job, FileRWLock* fileLock, BulletinBoardSystem* bbs,
           FileRWLock* messageLock);
    ~Worker();

    void workerMain();

private:

    //shared structs
    FileRWLock* fileLock;
    FileRWLock* messageLock;
    BulletinBoardSystem* bbs;

    //variables for secure protocol
    job_t* job;
    std::vector<uint8_t> iv;
    std::vector<uint8_t> hmacKey;
    std::vector<uint8_t> sessionKey;
    uint32_t counter;
    const std::string serverPrivateKeyPath = "res/keys/private/server.pem";

    //variables for user
    int userSocket;
    struct sockaddr_in userAddress;
    socklen_t userAddressLength;

    //thread logic functions
    void initiateProtocol();
    void waitForRequest();
    void AddHandler();
    void ListHandler();
    void GetHandler();

    //utility functions
    void sendAck();
    void sendError();
    ssize_t workerSend(const std::vector<uint8_t>& buffer);
    ssize_t receiveMessage(std::vector<uint8_t>& buffer, ssize_t bufferSize);
```

```

//functions to handle login and register
bool login();
bool registerUser();

bool checkUsername(const std::string& username);
bool checkEmail(const std::string& email);
void checkCounter(uint32_t counter);
bool writeUser(const std::string& username, const std::string& email, const
    uint8_t* hashedPassword);
bool checkPassword(const std::string& username, const uint8_t*
    hashedPassword);

};

```

2.2.1 Main functions

```
void workerMain()
```

This function aims to retrieve the request from the shared structure, and calls the functions to initiate the secure protocol and to handle the following requests from the client.

```

void Worker::workerMain(){
    while(true){
    {
        std::unique_lock<std::mutex> lock(job->mutex);
        job->cv.wait(lock, [&]() {return !job->queue.empty();});

        if(job->isDone)
            return;

        userSocket = job->queue.front();
        job->queue.erase(job->queue.begin());

    }
    try {
        initiateProtocol();
        waitForRequest();

    } catch (const std::exception &e) {
        std::cerr << e.what() << '\n';
    }
    }
}

```

The function logic can be described in these points:

1. In the main loop, a context for the lock is created, and a job (so a request) is retrieved from the shared structure `job`;
2. The target job is extracted from the head of the queue, assigning the right socket to the worker class variable;
3. At this point, worker doesn't need anymore to access the structure, and the lock is freed;
4. a try-catch block is used to attempt the call to the `initiateProtocol()` and `waitForRequest()` functions, permitting a more precise handling of possible errors in these functions.

```
void initiateProtocol()
```

Given that the communication protocol is the most crucial aspect of the application, we have opted to dedicate a separate chapter to it in this document. We will provide a detailed discussion of the design choices and its implementation, accompanied by relevant code snippets. Therefore, the code logic will be analyzed within that chapter for better understanding.

Functions `bool registerUser()` **and** `bool login()`

These functions have a standard approach to *sign-up* and *log-in* operations. However, it's worth to mention the usage of the `FileRWLock* fileLock` to guarantee a thread-safe access to the file which contains the users credentials.

- When a `FileRWLock::openForRead()` is called, the presence of a lock for writing operations is checked and, in that case, the thread will wait until that lock is freed. If any thread is performing a write operation, a read access is granted, allowing multiple threads to read the file at the same time.
- When a `FileRWLock::closeForRead()` call is performed, the read operation is finished and the number of readers is decremented. If the number of reader is equal to 0, and a thread is waiting for a write operation, that thread is notified that the read lock is now free.
- When a `FileRWLock::openForWrite()` operation is performed, the number of readers is checked, granting the write permission if the number of readers is null, and setting the write lock to prevent other threads to read while the write operation is being performed. In every other cases, thread will wait for a notification of read lock free.
- When a `FileRWLock::closeForWrite()` is called, the write lock is released, notifying every waiting readers.

Note that the choice to dedicate a chapter to the secure protocol is also applied to the generation of the challenge, used during the registration phase, so it will be discussed in the next chapter as well.

2.3 CLIENT

The class that describes the client actor is the following:

```
class Client {
public:
    Client(int port);
    ~Client();

    //connection and communication
    void connectToServer();
    void sendToServer(const std::vector<uint8_t>& message);
    void receiveFromServer(std::vector<uint8_t>& message);
    bool initiateProtocol(uint32_t mode);

    //login-register
    bool login();
    bool registerUser();

    //client logic
    void list(int n);
    void get(int mID);
    void add();

    std::string getUsername(){
        return username;
    }

private:
    int socket;
    int port;
    struct sockaddr_in serverAddress;
    std::string username;

    //secure protocol variables
    uint32_t counter;
    std::vector<uint8_t> hmacKey;
    std::vector<uint8_t> sessionKey;

    //utility functions
    void turnOnEcho();
    void turnOffEcho();
    void IncrementCounter();

    void sendPassword(std::string password);
};
```

3 SECURITY PROTOCOLS

3.0.1 Assumptions and Hypothesis

An assumption regards the parameters used for the key establishment: we are in the hypothesis that these parameters (the cyclic group and the corresponding primitive g) are fixed, and known to both client and server; we also assume that the pair $(ip, port)$ of the server are known by the client. Furthermore, we assume that the Server is able to retrieve the client public key K_{Cpub} correctly i.e. an attacker cannot perform a *Man-in-The-Middle* attack in order to substitute the K_{Cpub} with its own key.

3.1 SESSION ESTABLISHMENT PROTOCOL

The implemented protocol is an **authenticated key exchange** protocol, based on the well-known **Station-to-Station** protocol. The main idea behind the latter consists in a **Diffie-Hellman key establishment**, followed by the exchange of authentication signatures.

3.1.1 Protocol description

Protocol begins with the `client` creating a random number a , and sending the exponentiation g^a to the `server`, using a `struct ProtocolM1` message. It is then sent to the `server` that, upon receiving it, generate itself a random number b and compute both g^b and $K = g^{ab}$: while the first will be sent to the `client`, the latter will be the **shared secret**. The message `server` will send to the `client` is a `struct ProtocolM2` message, composed by its exponential and a *token*, which consists in the exponential, signed by the `server`, using AES in CBC mode. We also decided to hash the shared secret by using SHA-512: the 256 least significant bit are assigned to the session key K , while the 256 most significant ones are assigned to the `hmacKey` session key. Upon the `client` receives that message, it computes K and the `hmacKey`, then it decrypts the *token* using K , and finally verifies `server`'s signature using its public key K_{Spub} . At this point, `client` prepare its token, signing it using its private key K_{Cpriv} . It also specifies the *mode*, leading the `server` to prepare the operations for `registration` or `login`, depending on the value sent by `client`: these

variables are contained in a struct `ProtocolM3` message, sent by the client. Now server will receive the message, and it verifies the client signature by using K_{Cpub} . At this point, the establishment protocol is terminated, by having both client and server knowing the shared secret K and the `hmacKey`, and the operation of login/registration can begin.

3.1.2 Protocol implementation

To effectively implement the protocol, three different structures, one for each specific message, have been defined.

struct ProtocolM1

The struct is thus defined:

```
struct ProtocolM1 {
    uint32_t EPHkeyLength;
    uint8_t EPHKey[EPH_KEY_SIZE];
}
```

The variable `EPHKey[]` will contain the exponential g^a , while `EPHkeyLength` defines its length. Some utility methods have been define, such as the constructor, `serialize()` and `deserialize()` for sake of code readability, and `GetSize()` to retrieve the size of the message with ease.

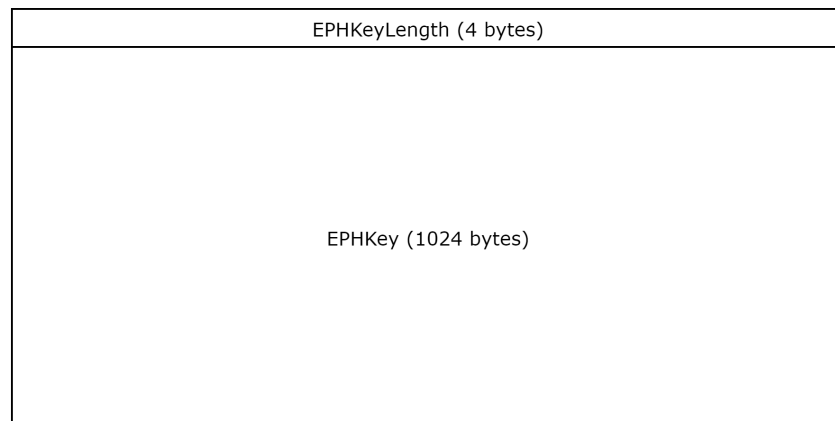


Figure 3.1. Structure of the message M1

The typical usage of the struct is shown in this snippet, taken from `Client::initiateProtocol()`:

```
//create the DiffieHellman object
DiffieHellman dh;
EVP_PKEY *EPH_KEY = nullptr;
```



```

try {
    EPH_KEY = dh.generateEPHKey();
}
catch (const std::exception &e) {
    ...
}

//serialize EPHEMERAL KEY
std::vector<uint8_t> serializedEPHKey;

try{
    serializedEPHKey = DiffieHellman::serializeKey(EPH_KEY);
} catch(const std::exception &e){
    ...
}

ProtocolM1 m1(serializedEPHKey, serializedEPHKey.size());

std::vector<uint8_t> serializedM1;
m1.serialize(serializedM1);

try{
    sendToServer(serializedM1);
} catch (const std::exception &e) {
    ...
}

```

struct ProtocolM2

The struct is thus defined:

```

struct ProtocolM2 {
    std::vector<uint8_t> EPHKey;
    uint32_t EPHKeyLength;
    std::vector<uint8_t> IV;
    uint32_t IVLength;
    std::vector<uint8_t> encryptedSignature;
    uint32_t encryptedSignatureLength;
}

```

Like the previous messages, the same utility methods have been defined.

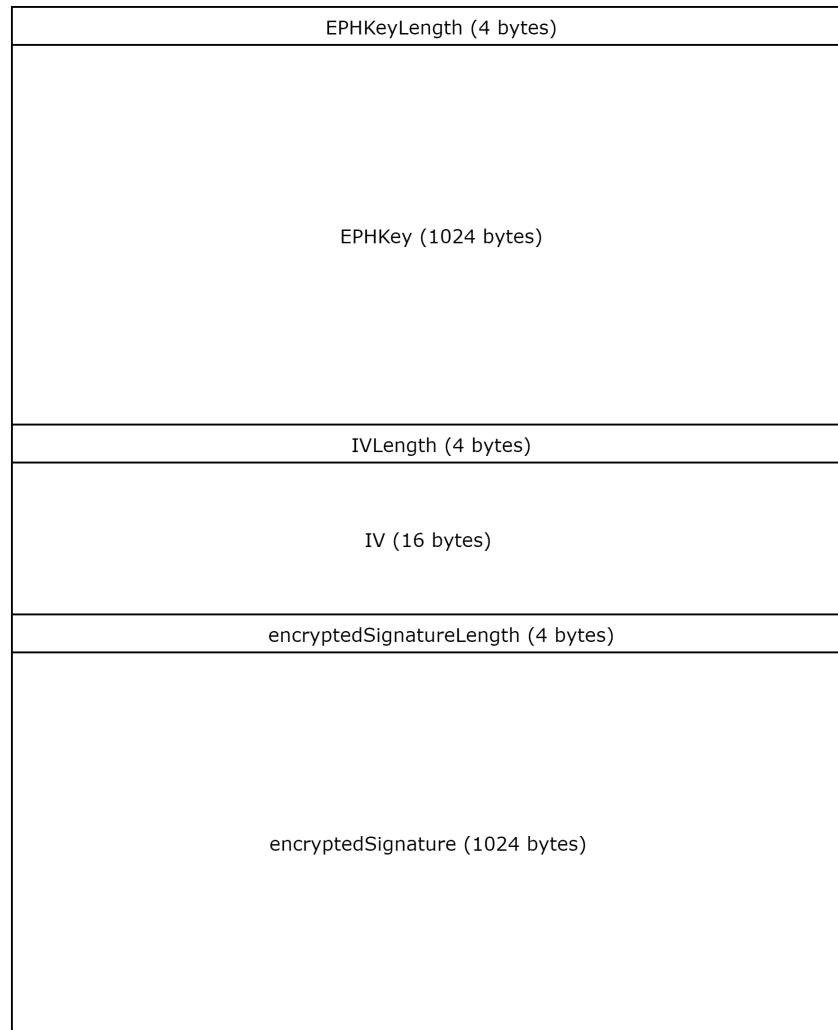


Figure 3.2. Structure of the message M2

The operations needed to correctly build the message and its content are showed in the following snippet, taken from `Worker::initiateProtocol()`:

```

DiffieHellman* dh = nullptr;
EVP_PKEY* EPH_KEY = nullptr;
EVP_PKEY* peerEPHKey = nullptr;

try{
    dh = new DiffieHellman();

    //generate EPH key
    EPH_KEY = dh->generateEPHKey();

    //deserialize peer EPH key
    peerEPHKey = DiffieHellman::deserializeKey(m1.EPHKey, m1.EPHkeyLength);

```

```

} catch (const std::exception &e) {
    ...
}
//generate shared secret
std::vector<uint8_t> sharedSecret;
size_t sharedSecretSize;

try {
    dh->generateSharedSecret(EPH_KEY, peerEPHKey, sharedSecret,
        sharedSecretSize);

    ...
} catch(const std::exception &e) {
    ...
}
//generate session hmac key
std::vector<uint8_t> keys;
uint32_t keySize;

try {
    SHA512::generateHash(reinterpret_cast<const unsigned
        char*>(sharedSecret.data()), sharedSecret.size(), keys, keySize);
    ...
} catch(const std::exception &e) {
    ...
}

std::memcpy(this->sessionKey.data(), keys.data(), (keys.size()/2) *
    sizeof(uint8_t));

std::memcpy(this->hmacKey.data(), keys.data() + ((keys.size()/2) *
    sizeof(uint8_t)), HMAC_DIGEST_SIZE * sizeof(uint8_t));

std::vector<uint8_t> serializedEPHKey;

try{
    serializedEPHKey = DiffieHellman::serializeKey(EPH_KEY);
}
catch(const std::exception &e){
    ...
}

EVP_PKEY_free(EPH_KEY);

auto EPHKeyBufferSize = m1.EPHkeyLength + serializedEPHKey.size();
std::vector<uint8_t> EPHKeyBuffer(EPHKeyBufferSize);

```

```

std::memcpy(EPHKeyBuffer.data(), m1.EPHKey, m1.EPHkeyLength);
std::memcpy(EPHKeyBuffer.data() + m1.EPHkeyLength, serializedEPHKey.data(),
    serializedEPHKey.size());

std::vector<unsigned char> signature;
RSASignature* rsa = nullptr;

try {
    rsa = new RSASignature(serverPrivateKeyPath, "");
    signature = rsa->sign(EPHKeyBuffer);
} catch (const std::exception &e) {
    ...
}

std::vector<uint8_t> iv(EVP_CIPHER_iv_length(EVP_aes_256_cbc()));
std::vector<uint8_t> cipherText;

AESCBC* enc = nullptr;

try {
    enc = new AESCBC(ENCRYPT, sessionKey);
    enc->run(signature, cipherText, iv);

    ...
} catch (const std::exception &e) {
    ...
}

try {
    ProtocolM2 m2(serializedEPHKey, iv, cipherText);
    std::vector<uint8_t> serializedM2 = m2.serialize();

    workerSend(serializedM2);

    ...
} catch (const std::exception &e) {
    ...
}

```

struct ProtocolM3

The struct is thus defined:

```

struct ProtocolM3 {
    uint32_t iv_length;
    std::vector<uint8_t> iv;
    uint32_t encSignatureSize;

```

```
std::vector<uint8_t> encSignature;
uint32_t mode;}
```

Utility function such as the one for the previous struct have been defined also for ProtocolM3.

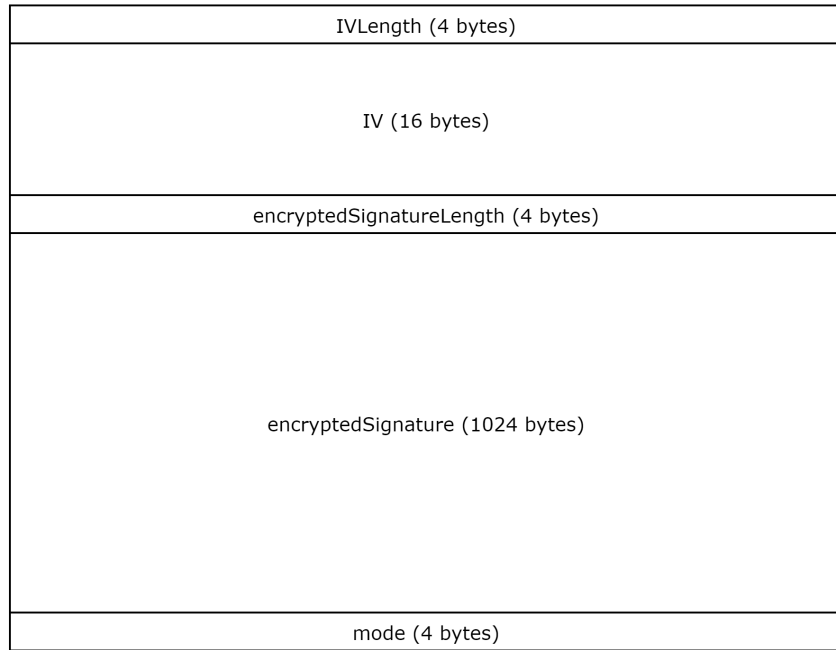


Figure 3.3. Structure of the message M3

Operations needed to build a correct message are those in the `Client::initiateProtocol()`:

```
EVP_PKEY *serverEPHKey = nullptr;
std::vector<uint8_t> sharedSecret;

try {
    serverEPHKey = DiffieHellman::deserializeKey(m2.EPHKey.data(),
        m2.EPHKeyLength);
    dh.generateSharedSecret(EPH_KEY, serverEPHKey, sharedSecret,
        sharedSecret.size());
} catch (const std::exception &e) {
    ...
}
std::vector<uint8_t> keys;
uint32_t keySize;

try {
    SHA512::generateHash(sharedSecret.data(), sharedSecret.size(), keys,
        keySize);
```

```

} catch (const std::exception &e) {
    ...
}

std::memcpy(this->sessionKey.data(), keys.data(), (keys.size() / 2) *
    sizeof(uint8_t));
std::memcpy(this->hmacKey.data(), keys.data() + (keys.size() / 2)*
    sizeof(uint8_t), HMAC_DIGEST_SIZE * sizeof(uint8_t));

//prepare <g^a, g^b>
int EPHKeyBufferLength = m2.EPHKeyLength + serializedEPHKey.size();
std::vector<uint8_t> EPHKeyBuffer(EPHKeyBufferLength);
std::memcpy(EPHKeyBuffer.data(), serializedEPHKey.data(),
    serializedEPHKey.size());
std::memcpy(EPHKeyBuffer.data() + serializedEPHKey.size(), m2.EPHKey.data(),
    m2.EPHKeyLength);

std::vector<uint8_t> signature;
try {
    char privateKeyPath[] = "res/keys/private/client1.pem";
    RSASignature rsa(privateKeyPath, "");
    signature = rsa.sign(EPHKeyBuffer);
} catch (const std::exception &e) {
    ...
}

std::vector<uint8_t> iv(EVP_CIPHER_iv_length(EVP_aes_256_cbc()));
std::vector<uint8_t> encryptedSignature;
AESCBC* encryptor = nullptr;

try {
    encryptor = new AESCBC(ENCRYPT, this->sessionKey);
    encryptor->run(signature, encryptedSignature, iv);
} catch (const std::exception &e) {
    ...
}

std::vector<uint8_t> decryptedSignature;
AESCBC* decryptor = nullptr;

try {
    decryptor = new AESCBC(DECRYPT, this->sessionKey);
    decryptor->run(m2.encryptedSignature, decryptedSignature, m2.IV);
} catch (const std::exception &e) {
    ...
}

//verify the signature
RSASignature* rsa = nullptr;

```

```

bool verified = true;

try {
    //retrieve the public key of the server
    char publicKeyPath[] = "res/keys/public/server.pem";

    rsa = new RSASignature("", publicKeyPath);
    verified = rsa->verify(EPHKeyBuffer, decryptedSignature);

} catch (const std::exception &e) {
    ...
}

if (!verified) {
    throw std::runtime_error("Signature not verified");
}
ProtocolM3 m3(iv, encryptedSignature, mode);
std::vector<uint8_t> serializedM3 = m3.serialize();

try{
    sendToServer(serializedM3);

} catch (const std::exception &e) {
    ...
}

```

Protocol schema

The flowchart of the Session Establishment Protocol is the following

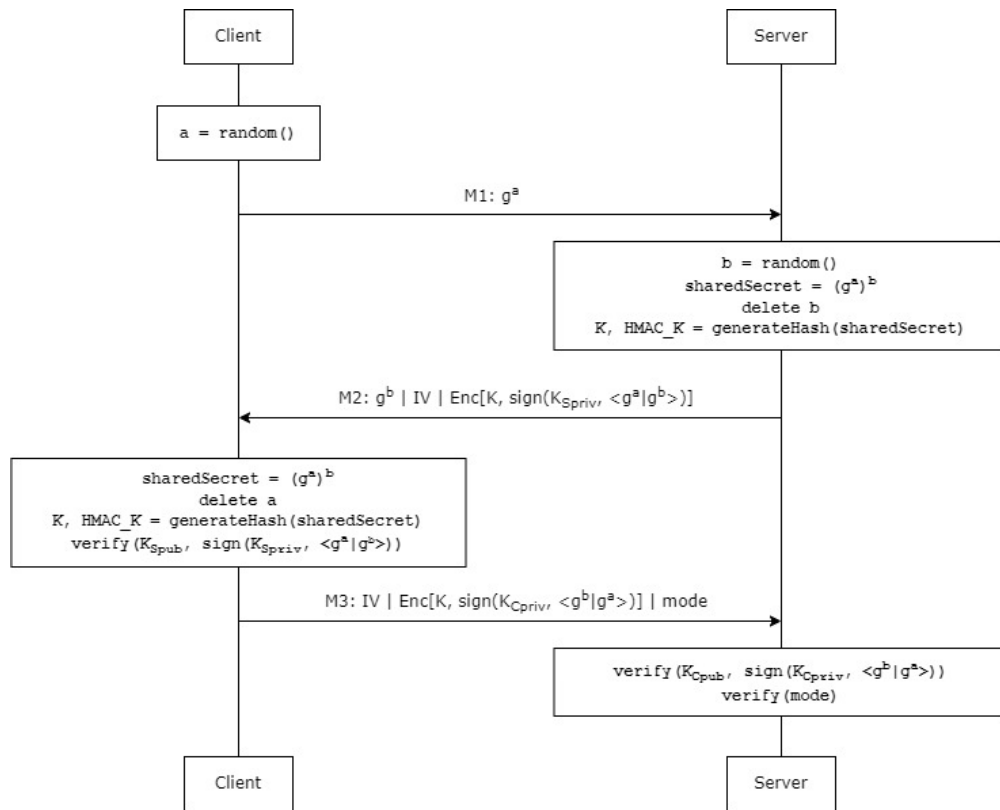


Figure 3.4. Flowchart of the Session Establishment Protocol

Security consideration

From server's point of view, as a result of DH key exchange, it shares a key known only to it and the other party, which could not be the client. Given the fact that it signed the exponential associated with the specific connection, one of which has been created by server for that very connection, its signature is tied to the specific connection and run of the protocol. By encrypting its signature with K , client demonstrates to server that it was the actual part that generates g^a . It gets a similar assurance from server, because it has signed the exponents specific of the very run, one of which has been created by the verified client. Thus, encrypting its signature with K , server demonstrates to client that it was the actual part who generates g^b .

Thus, the authentication is **direct**, and this because both client and server show knowledge of the exchanged key by encrypting their signatures. Moreover, we can say that the protocol offers **Perfect Forward Secrecy**; the only long-term secret is the private key of each part, used in the signature schema: if the secret key is compromised, the security of the keys exchanged in previous runs is not affected thanks to the Diffie-Hellman protocol.

Speaking of protections against **replay attacks**, a counter has been implemented; every time a message is sent across the channel to the other part, the counter is incremented, with receiver checking for replays attack by comparing the received counter with its own value: if check is

passed, receiver updates its own counter. The usage of that counter, there exists the possibility of **wrap around** of the counter value; however, this condition is checked every time the counter is incremented. In case of both replay attack or wrap around, the socket is closed and every structure used is deleted, thus a new establishment protocol is needed. This choose to interrupt the communication even if a wrap around occurs was driven by the fact that wrap around is a unlikely event to happen, compared to the average number of messages exchanged from the parts within a session.

3.2 SESSION MESSAGE PROTOCOL

The encryption method chosen for each message within a session is AES in CBC mode. Using such cipher the system provide **confidentiality**. The structure of a session message is the following:

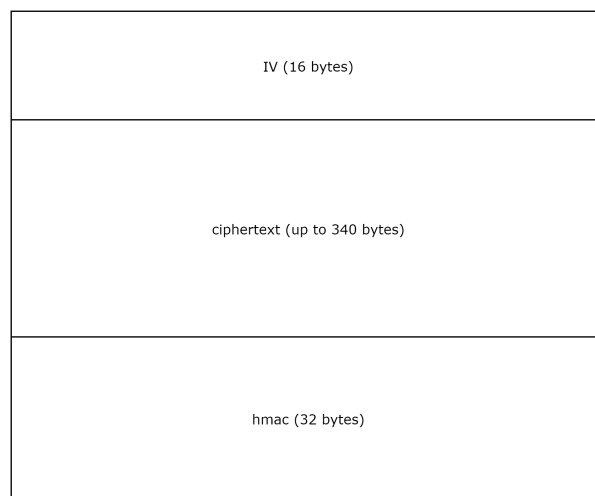


Figure 3.5. Structure of the sessionMessage

3.2.1 HMAC generation protocol

An HMAC approach was chosen to ensure **integrity** and **non-malleability**. The generation schema is the following

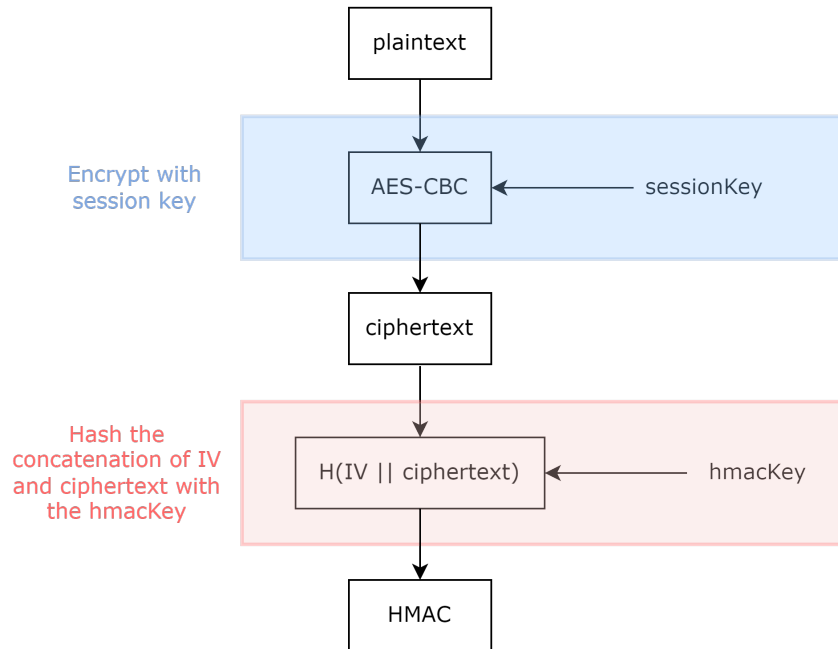


Figure 3.6. Flowchart of the HMAC generation

3.2.2 Message structure

The class that implements the session message is the following:

```

struct sessionMessage {
    std::vector<uint8_t> iv;
    std::vector<uint8_t> ciphertext;
    std::vector<uint8_t> hmac;
}

```

Utility functions, such as `serialize()`, `deserialize()`, `verify_HMAC()` and `decrypt()` have been implemented.

The message schema is in [Figure 3.5](#).

Constructor

The constructor performs the encryption and the computation of the HMAC:

```

sessionMessage::sessionMessage(const std::vector<uint8_t>& session_key, const
    std::vector<uint8_t>& hmac_key, const std::vector<uint8_t>& plaintext){
    iv.resize(EVP_CIPHER_iv_length(EVP_aes_256_cbc()));
    AESCBC encrypt(ENCRYPT, session_key);
    encrypt.run(plaintext, ciphertext, iv);
}

```

```

//concatenate iv and ciphertext
std::vector<uint8_t> buffer(iv.size() + ciphertext.size());
std::copy(iv.begin(), iv.end(), buffer.begin());
std::copy(ciphertext.begin(), ciphertext.end(), buffer.begin() + iv.size());

//generate HMAC
HMac hmac(hmac_key.data());
std::vector<uint8_t> digest;
unsigned int digest_size = 0;
hmac.generate(buffer.data(), iv.size() + ciphertext.size(), digest,
    digest_size);
this->hmac.resize(digest_size);
std::copy(digest.begin(), digest.end(), this->hmac.begin());

std::memset(buffer.data(), 0, buffer.size()*sizeof(uint8_t));
buffer.clear();

std::memset(digest.data(), 0, digest.size()*sizeof(uint8_t));
digest.clear();
}

```

3.2.3 Buffer length management

A session message can be used to only send operation requests (i.e. an integer which corresponds to an operation), a bbs message, the password or the credentials (username and/or email), thus the ciphertext length is fixed and known to both `client` and `server` in any moment. Because of this implementation, we don't need to send a message with the length of the incoming actual message, thus we can instead send immediately the actual message. In particular:

- if a **request** or a **response** is sent, the size is fixed to the length of an encryption of a `uint_32` variable;
- if a **bbs message** is sent, the size is fixed to the maximum size available for that field. Padding with `0x00` will be performed for those strings that are shorter than the maximum length.
- if the **password** is sent, the size is fixed to the length of the computed hash;
- if a **credential** is sent, the size is fixed because of the padding added to reach the maximum size.

3.3 LOGIN PHASE PROTOCOL

The login is performed by `client` sending to `server` two messages, one containing the username and one containing the password.

The flowchart of the login operation is the following:

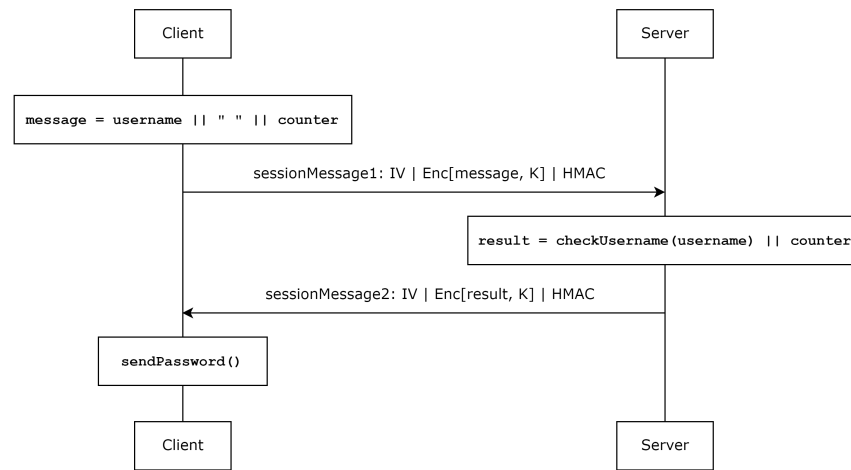


Figure 3.7. Flowchart of the login

3.3.1 Password management

A function to send the password to the server has been implement:

```

void Client::sendPassword(std::string password) {
    IncrementCounter();

    PasswordMessage message(password.c_str(), this->counter);

    std::vector<uint8_t> plaintext(PWD_MESSAGE1_SIZE);
    message.serialize(plaintext);

    sessionMessage sessionMessage(this->sessionKey, this->hmacKey, plaintext);

    std::memset(plaintext.data(), 0, plaintext.size());
    plaintext.clear();

    std::vector<uint8_t> serializedSessionMessage = sessionMessage.serialize();

    try {
        sendToServer(serializedSessionMessage);
    } catch (const std::exception &e) {
        ...
    }
    return;
}
  
```

Before becoming the ciphertext of a `sessionMessage`, the plain password is processed by the `PasswordMessage`, of which we can see the constructor in the following snippet:

```
PasswordMessage(const char* password, uint32_t counter) {
    //data sanitization of the password string
    if (strlen(password) > PASSWORD_MAX_SIZE) {
        throw std::invalid_argument("Password is too long");
    }
    //check for whitespaces, special characters, endlines, etc
    for (int i = 0; i < strlen(password); i++) {
        if (password[i] < 32 || password[i] > 126) {
            printf("Invalid character: %c\n", password[i]);
            throw std::invalid_argument("Invalid character in password");
        }
    }

    memset(&this->password, 0, HASHED_PASSWORD_SIZE);
    std::vector<uint8_t> hash(HASHED_PASSWORD_SIZE);
    unsigned int digestLength = HASHED_PASSWORD_SIZE;
    SHA512::generateHash((const unsigned char*)password, strlen(password),
        hash, digestLength);
    std::memcpy(this->password, hash.data(), HASHED_PASSWORD_SIZE);
    this->counter = counter;
}
```

The logic behind the entire of password sending can be summarized in these point:

1. password is collected from `std::cin`, with echo turned off, and the `sendPassword()` is called;
2. in that function, a `passwordMessage` variable is declared: the constructor performs the hash of the password and copy the buffer in the new object;
3. a `sessionMessage` variable is declared, with the `passwordMessage` as the plaintext. The `sessionMessage` is appropriately initialized, and then sent to server.

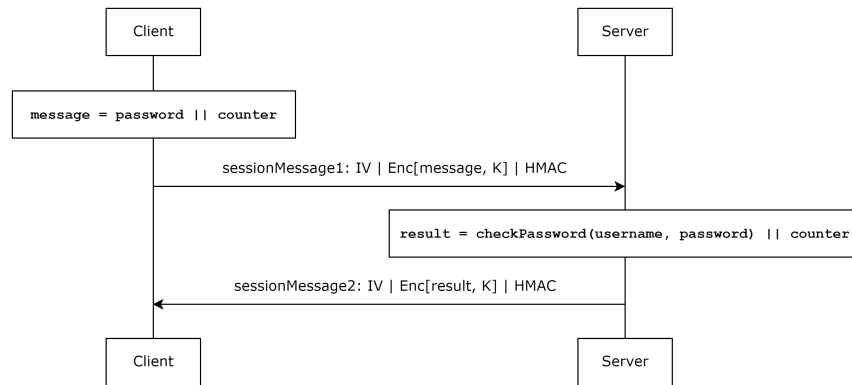


Figure 3.8. Flowchart of the sending password operation

3.4 REGISTRATION PHASE PROTOCOL

The flowchart of the registration is the following:

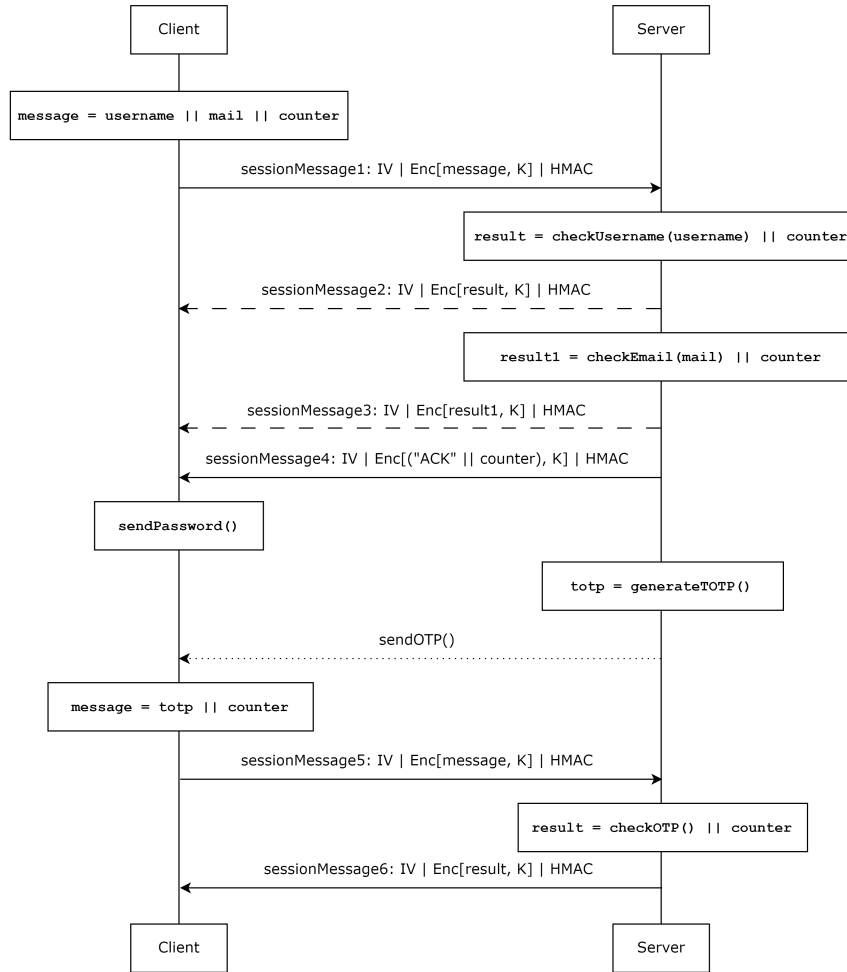


Figure 3.9. Flowchart of the registration

3.4.1 Challenge generator

To handle the need of a challenge to verify the user before finalizing the registration phase, the TOTPGenerator class has been defined. Its only private member is the `secretKey_` which has been generated by the server by using OPENSSL, following the RFC 6238 standard. The function that generates the challenge is the following:

```

std::string TOTPGenerator::generateTOTP(uint64_t timeStep) const {
    time_t now = time(nullptr);
    uint64_t timeCounter = now / timeStep;

    std::vector<uint8_t> counterBytes(sizeof(timeCounter));
    for (ssize_t i = 0; i < sizeof(timeCounter); ++i) {
        counterBytes[sizeof(timeCounter) - i - 1] = (timeCounter >> (i * 8)) &
            0xFF;
    }
}
  
```

```

EVP_MD_CTX* ctx = EVP_MD_CTX_new();
if (ctx == nullptr) {
    throw std::runtime_error("Failed to create EVP_MD_CTX");
}

if (EVP_DigestInit_ex(ctx, HMAC_HASH(), nullptr) != 1) {
    EVP_MD_CTX_free(ctx);
    throw std::runtime_error("Failed to initialize digest");
}

if (EVP_DigestUpdate(ctx, secretKey_.c_str(), secretKey_.size()) != 1) {
    EVP_MD_CTX_free(ctx);
    throw std::runtime_error("Failed to update digest");
}

if (EVP_DigestUpdate(ctx, counterBytes.data(), counterBytes.size()) != 1) {
    EVP_MD_CTX_free(ctx);
    throw std::runtime_error("Failed to update digest");
}

uint8_t hash[EVP_MAX_MD_SIZE];
unsigned int hashLength;

if (EVP_DigestFinal_ex(ctx, hash, &hashLength) != 1) {
    EVP_MD_CTX_free(ctx);
    throw std::runtime_error("Failed to finalize digest");
}

EVP_MD_CTX_free(ctx);

uint32_t offset = hash[hashLength - 1] & 0x0F;
uint32_t binary = (hash[offset] & 0x7F) << 24 | (hash[offset + 1] & 0xFF) <<
    16 | (hash[offset + 2] & 0xFF) << 8 | (hash[offset + 3] & 0xFF);

uint32_t otp = binary % 1000000;

//if the otp is less than 6 digits, add zeros to the left
if (otp < 100000) {
    return "0" + std::to_string(otp);
}

return std::to_string(otp);
}

```

We can resume its logic in four points:

1. **Time-based Counter:** Retrieves the current time and calculates a counter based on the provided time step (default 30 seconds). This counter reflects how many time steps have passed since a specific point, determining the window of validity for the generated OTP.
2. **Secure Hashing with Secret Key:** Utilizes the OpenSSL library to perform HMAC-SHA512. It feeds the secret key stored in the class and the time counter byte array into the hashing process. This ensures the generated OTP is unique and unpredictable, relying on the secrecy of the key.
3. **OTP Extraction from Hash:** Finalizes the HMAC-SHA512 digest, resulting in a hash value. A specific section of this hash is extracted based on a calculated offset. This portion is then converted into a 32-bit unsigned integer, forming the core of the OTP value.
4. **Formatting OTP as 6-Digit String:** Converts the 32-bit OTP value into a string representation. It guarantees the OTP is a 6-digit code by adding a leading zero if necessary, ensuring a consistent user experience when displaying the OTP.

4 APPLICATION PROTOCOLS

4.1 LIST() OPERATION

The following figure show the flowchart of the `list()` operation:

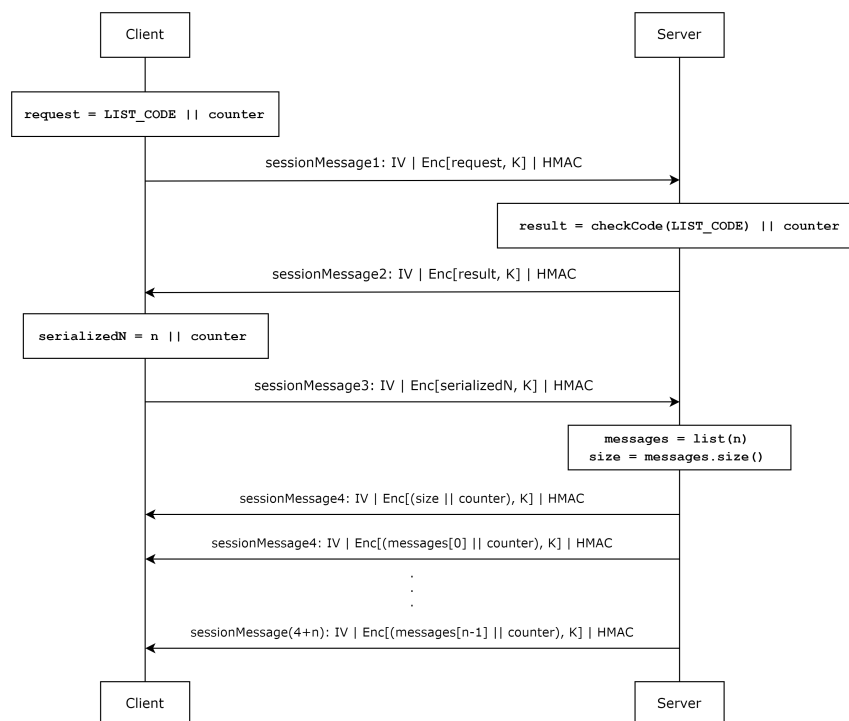


Figure 4.1. Flowchart of the `list()` operation

The edge case when `n` results higher of the actual number of messages in the system is handled by the server sending, before the actual messages, the number of messages it'll send, letting know the client the number of messages it should receive.

4.2 ADD() OPERATION

The flowchart of the function logic is the following:

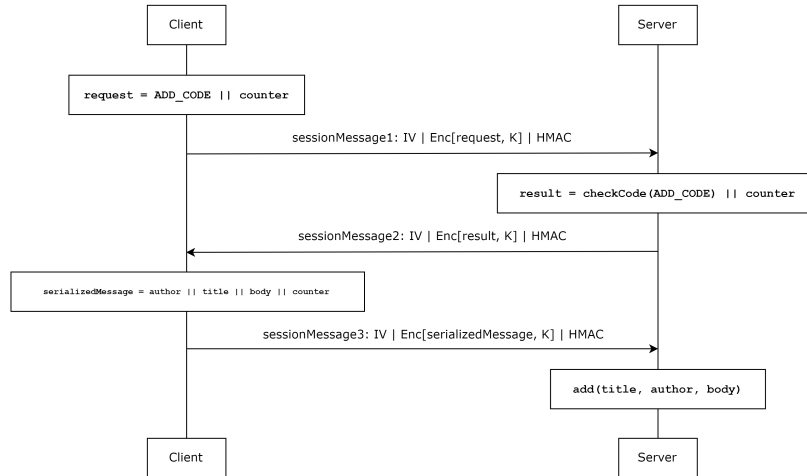


Figure 4.2. Flowchart of the `add()` operation

4.3 GET() OPERATION

The flowchart of the function logic is the following:

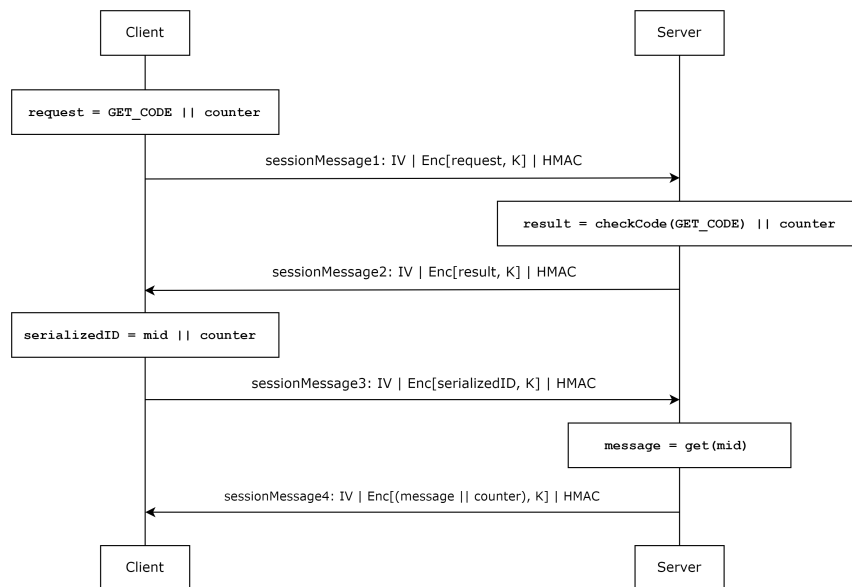


Figure 4.3. Flowchart of the `get()` function

In case there is not a message with the specified ID, the server will respond with an message that is empty, except for the ID, whose value is 0. This is possible because in our implementation, ID values starts from 1, so ID=0 is a well-know value that indicates an empty message, and an appropriate message will be shown.